14

Transaktionsverwaltung

14.1 Konsistente Datenbestände und Transaktionen

Datenbanksysteme erlauben den gleichzeitigen Datenzugriff mehrerer Benutzer. Bei lesendem Zugriff auf die Daten ist dies kein Problem. Wenn jedoch mehrere Benutzer zur selben Zeit die gleichen Datensätze bearbeiten, muss das Datenbanksystem für die Konsistenz der Daten sorgen.

Darüber hinaus muss die Datenkonsistenz auch bei Fehlern gewährleistet werden, die z. B. durch Programmfehler oder Hardwareausfälle auftreten können. Das Datenbanksystem muss daher in der Lage sein, die betroffenen Daten in jedem Fall in den letzten konsistenten Zustand zu überführen. Zu diesem Zweck verwenden die meisten Systeme die sogenannte Transaktionsverwaltung.

Transaktionen

Transaktionen sind eine Gruppe von logisch zusammenhängenden Datenbankoperationen (SQL-Anweisungen), die nur **gemeinsam** ausgeführt werden sollen. Kann auf Grund eines Fehlers, einer Zugriffsverletzung oder Ähnlichem eine der Operationen nicht ausgeführt werden, dann wird keine der Operationen in der Transaktion ausgeführt und der Datenbestand wird in den Ausgangszustand versetzt.

Das Datenbanksystem garantiert bei der Ausführung einer Transaktion die Einhaltung der vier grundlegenden Eigenschaften, die auch als ACID-Eigenschaften bezeichnet werden:

Atomicity – Atomarität 'Alles oder Nichts'	Die Transaktion, die aus einer oder mehreren Operationen bestehen kann, wird entweder vollständig ausgeführt oder gar nicht. Tritt während der Transaktion ein Fehler auf (z. B. Programmfehler, Hardwarefehler oder Betriebssystemabsturz), werden alle bisherigen Operationen der Transaktion rückgängig gemacht.

Schulversion
© HERDT-Verlag

Consistency – Konsistenz	Ist die Transaktion abgeschlossen, befindet sich die Datenbank in einem konsistenten Zustand. Dazu tragen die definierten **Integritäts-bedingungen** (Wertebereiche, Schlüsseleigenschaften, Fremdschlüssel usw.) bei, welche die logische Konsistenz sichern. Verletzungen dieser Integritätsbedingungen führen zum Zurücksetzen der Transaktion. Während einer Transaktion können aber zwischenzeitlich inkonsis-tente Zustände auftreten, die mit einer weiteren Operation wieder-hergestellt werden. Solche „verzögerten Integritätsbedingungen“ treten beispielsweise bei der Umbuchung zwischen Konten auf. Sie werden meist am Ende der Transaktion überprüft.
Isolation – Isolation	Transaktionen laufen isoliert ab, mehrere gleichzeitig ablaufende Transaktionen stören und beeinträchtigen sich nicht gegenseitig. Das wird seitens des DBMS durch geeignete **Synchronisations-maßnahmen** erreicht. Dies ist eine wichtige Voraussetzung zur Sicherung der Konsistenz der Datenbank.
Durability – Dauerhaftigkeit	Das Ergebnis einer erfolgreichen Transaktion ist dauerhaft (persistent). Das bedeutet, dass vom DBMS sichergestellt wird, dass auch beim Auftreten von Fehlern bei der Übertragung der Transaktionsergebnisse in die Datenbank die Änderungen vollständig durchgeführt werden.

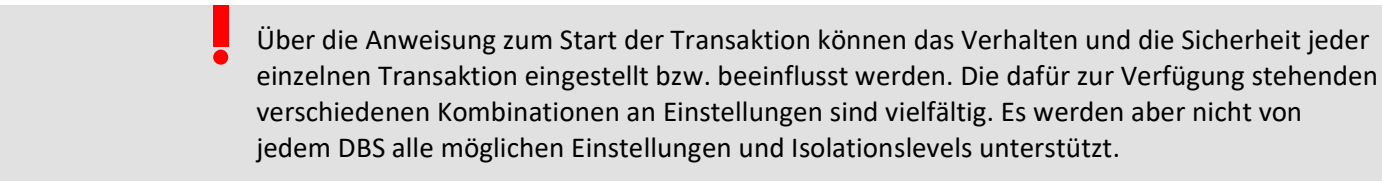
Isolationsebenen (Transaction Isolation Level)

Durch den Transaction Isolation Level wird der Grad der Parallelität von Transaktionen gesteuert. Je höher Sie den Level wählen, desto sicherer können Benutzer sein, dass sie bei einer Leseaktion die aktuellen Daten erhalten, und umso weniger Probleme bezüglich der Konsistenz können auftreten. Je höher Sie aber die Isolationsebenen wählen, desto geringer ist der Durchsatz bei einer großen Zahl gleichzeitiger Zugriffe, was durch die notwendigen zeitweiligen Sperrungen der Daten hervorgerufen wird. Im SQL2-Standard werden vier Transaction Isolation Levels definiert.

Isolationsebene 0: READ UNCOMMITTED	Auf Daten, die von einer Transaktion geschrieben werden, kann mittels einer anderen Transaktion lesend zugegriffen werden. Dabei können zum Teil neue und veraltete Daten gelesen werden = unsauberes Lesen (dirty read). Von der Transaktion werden keine Sperren auf die Datensätze gesetzt.
Isolationsebene 1: READ COMMITTED	Datensätze, die von der Transaktion modifiziert werden, sind gesperrt, Datensätze, die nur gelesen wurden, aber nicht. Dadurch können die Datensätze, die nur gelesen wurden, zwischenzeitlich geändert werden. Das Problem des inkonsistenten Lesens und das sogenannte Phantome-Problem können auftreten.
Isolationsebene 2: REPEATABLE READ	Die Transaktion sperrt alle Datensätze, die von ihr gelesen und geschrieben werden. Nur das Phantome-Problem kann hier noch auftreten.
Isolationsebene 3: SERIALIZABLE	Damit wird sichergestellt, dass in einer Transaktion gelesene Daten bis zum Ende der Transaktion gültig sind. Das Lesen von sogenannten Phantomen wird verhindert. Diese Ebene bietet volle Serialisierbarkeit.

© HERDT-Verlag Schulversion

Phantome sind ausgewählte Daten, die einmal von einer Transaktion A gelesen wurden und anschließend durch Schreib-, Änderungs- oder Lösch-Operationen in einer anderen Transaktion B geändert werden. Die gleiche Leseoperation der ersten Transaktion A würde danach eine andere Auswahl von Daten bringen. Transaktion A arbeitet mit veralteten Daten. Beispielsweise werden in Transaktion A alle Artikel ausgewählt, von denen weniger als 1 Stück im Lager ist. In Transaktion B werden 2 dieser Artikel gelöscht. Bei der gleichen Anfrage würde Transaktion A nun 2 Datensätze weniger erhalten. Sie arbeitet nun mit Daten, die eigentlich nicht mehr vorhanden sind.



Über die Anweisung zum Start der Transaktion können das Verhalten und die Sicherheit jeder einzelnen Transaktion eingestellt bzw. beeinflusst werden. Die dafür zur Verfügung stehenden verschiedenen Kombinationen an Einstellungen sind vielfältig. Es werden aber nicht von jedem DBS alle möglichen Einstellungen und Isolationslevels unterstützt.

Beispiel für eine Transaktion: *Transaktionen.sql*

Für eine Aktion sollen alle Artikel, von denen mehr als 200 Stück am Lager sind, um 5 % verbilligt angeboten werden. Die Artikel, von denen zwischen 100 und 200 Stück vorrätig sind, werden um 2 % billiger.

Diese Operationen sind entweder für alle oder für keinen Artikel auszuführen. Eine Wiederholung der Operation würde auch bereits geänderte Preise nochmals ändern.

```
① SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
② UPDATE t_lager SET preis = preis * 0.95 WHERE stueck > 200;
  UPDATE t_lager SET preis = preis * 0.98
    WHERE stueck > 100 and stueck <= 200;
③ COMMIT;
```

① Mit dieser Anweisung wird eine neue Transaktion gestartet. Die Option `ISOLATION LEVEL REPEATABLE READ` steuert das Verhalten des Datenbanksystems während der Transaktion. In diesem Fall wird das Isolationslevel 2 gesetzt.

② Hier erfolgen die beiden `UPDATE`-Anweisungen zum Ändern der Preise in der Tabelle *t_lager*.

③ Mit der Anweisung `COMMIT` wird die begonnene Transaktion bestätigt und die Änderungen werden endgültig ausgeführt.

14.2 Transaktionen erstellen

Transaktionen können mit der `START-TRANSACTION`- oder der `SET-TRANSACTION`-Anweisung gestartet werden. Bei der zweiten Version haben Sie die Möglichkeit, mit verschiedenen Klauseln zu steuern, wie sich das DBMS während der Transaktion in Bezug auf Zugriffe anderer Transaktionen auf die gleichen Tabellen verhalten soll. Zusätzlich ist es möglich, nur lesende Zugriffe oder lesende und schreibende Zugriffe für die Anweisungen der Transaktion zuzulassen.

© HERDT-Verlag

Sie können bei MariaDB und PostgreSQL für das Starten einer Standard-Transaktion die Anweisung BEGIN ohne weitere Parameter verwenden.

```
BEGIN;
```

Syntax der SET-TRANSACTION-Anweisung

```
SET TRANSACTION [transaktionsname] [READ WRITE|READ ONLY]
  [WAIT|NO WAIT] [ISOLATION LEVEL SERIALIZABLE| REPEATABLE READ |
  READ COMMITTED| READ     UNCOMMITTED];
```

- Die Anweisung beginnt mit den Schlüsselwörtern SET TRANSACTION. Danach kann ein Transaktionsname angegeben werden.
- Über die Schlüsselwörter READ WRITE oder READ ONLY können Sie steuern, für welche Zugriffe die Sperrung der Tabellen für die Operationen während der Transaktion erfolgen soll (nicht bei MariaDB).
- Dann kann über die Klausel WAIT bzw. NO WAIT das Warteverhalten der Transaktion in Bezug auf andere Prozesse beeinflusst werden (nicht bei MariaDB und PostgreSQL).
- Optional kann durch die Klausel ISOLATION LEVEL, gefolgt von den entsprechenden Schlüsselwörtern, angegeben werden, welche Isolationsebene für die Transaktion eingestellt werden soll.

Zugriffsart für Tabellen festlegen

Mit den Optionen READ WRITE und READ ONLY können Sie festlegen, ob während der Transaktion alle Zugriffsarten oder nur lesende Zugriffe auf die Daten ausgeführt werden sollen.

READ WRITE	Die Operationen der Transaktion können Daten aus den betreffenden Tabellen lesen und Datensätze in den Tabellen ändern, löschen und neu einfügen.
READ ONLY	Die Operationen der Transaktion dürfen nur Daten aus den Tabellen mit der SELECT-Anweisung lesen.

Die READ-WRITE-Option ist die Standardeinstellung. Diese Angabe kann daher auch entfallen.

Wenn Sie eine Transaktion für schreibende Zugriffe sperren, erzeugt das Ausführen einer INSERT-, UPDATE- oder DELETE-Anweisung eine Fehlermeldung. Wird eine solche Transaktion beispielsweise in einem Anwendungsprogramm eingesetzt, so können Sie damit das unerlaubte oder unbewusste Ändern von Daten durch den Benutzer des Programms unterbinden.

Das Datenbanksystem MariaDB unterstützt die Optionen READ WRITE und READ ONLY nicht. Dafür kann durch die Anweisung LOCK TABLES das Lese- bzw. Schreibrecht für den aktuellen Thread eingeräumt werden. Beispielsweise kann auf die Tabelle *t_lager* nur noch lesend zugegriffen werden, wenn sie durch die Anweisung

```
LOCK TABLE t_lager READ;
```

gesperrt wird. Durch den Aufruf von UNLOCK TABLES wird diese Sperrung wieder aufgehoben.

Warten auf andere Prozesse

Mit den Optionen `WAIT` und `NO WAIT` steuern Sie das Verhalten der Transaktionen, wenn andere Transaktionen gerade auf die gleichen Daten zugreifen.

`WAIT`	Wird festgestellt, dass eine andere Transaktion auf die gleichen Tabellen zugreift, wird mit den Operationen der Transaktion gewartet, bis alle anderen Zugriffe beendet wurden. Dadurch werden Zugriffskonflikte vermieden.
`NO WAIT`	Beim gleichzeitigen Zugriff anderer Transaktionen auf die an der Transaktion beteiligten Tabellen wird sofort eine Fehlermeldung erzeugt und die Transaktion wird abgebrochen.

Die Option `WAIT` ist die Standardeinstellung. Sie kann daher auch entfallen.

Die Option `NO WAIT` ist sinnvoll, wenn Sie z. B. bei Lesezugriffen auf eine Tabelle sofort erfahren möchten, dass die Abfrage möglicherweise ungültige und nicht mehr aktuelle Daten liefert.

Unter MariaDB und PostgreSQL sind die Optionen `WAIT` und `NO WAIT` nicht verfügbar.

Isolation Level

Mit der Option `ISOLATION LEVEL` können Sie beim Erstellen einer Transaktion steuern, wie diese von anderen Transaktionen isoliert werden soll. Die Isolation der Daten bei konkurrierenden Transaktionen auf den gleichen Datenbestand ist ein wichtiges Merkmal vieler relationaler Datenbanksysteme.

Bei der Isolation der Daten entsteht das Problem, dass einerseits der parallele Zugriff mehrerer Benutzer unbedingt erwünscht ist, andererseits jedoch die Konsistenz der Daten durch isolierte Transaktionen bewahrt werden soll. Durch eine zu starke Isolation verliert ein Datenbanksystem seine Tauglichkeit für die Nutzung durch sehr viele parallele Benutzerzugriffe. Die meisten Datenbanksysteme verfügen daher über unterschiedliche Implementierungen der Klausel `ISOLATION LEVEL`. Mehr Informationen darüber finden Sie im Hilfesystem Ihrer Datenbank.

Im Folgenden sind einige Klauseln aufgeführt. Diese können in verschiedenen DBS mit unterschiedlichen Schlüsselwörtern definiert sein.

`ISOLATION LEVEL READ UNCOMMITTED`
oder
`ISOLATION LEVEL SNAPSHOT`
	Zu Beginn der Transaktion werden die aktuellen Daten kopiert; es wird ein Schnappschuss davon erzeugt. Dieser Schnappschuss dient als Grundlage für die Operationen der Transaktion. Änderungen, die durch andere gleichzeitig laufende Transaktionen durchgeführt werden, sind im Datenbestand dieser Transaktion nicht sichtbar (Isolationsebene 0).

© HERDT-Verlag

`ISOLATION LEVEL READ COMMITTED`	Bei Angabe dieser Klausel können im Verlauf der Transaktion auch Änderungen durch andere Transaktionen stattfinden, wenn diese mit `COMMIT` abgeschlossen wurden. Für die Transaktion sind diese Änderungen sichtbar (Isolationsebene 1).

`ISOLATION LEVEL REPEATABLE READ`
oder
`ISOLATION LEVEL SNAPSHOT TABLE STABILITY`
	Bei diesem Isolation Level wird sichergestellt, dass die an der Transaktion beteiligten Tabellen für Schreibzugriffe anderer Transaktionen gesperrt sind. Andere Transaktionen können aber trotzdem daraus lesen (Isolationsebene 2).

`ISOLATION LEVEL SERIALIZABLE`	Dieser Isolation Level gewährt die volle Serialisierbarkeit (Isolationsebene 3).

In einigen SQL-Dialekten wird für die Wahl des `ISOLATION LEVEL` nur die entsprechende Nummer angegeben.

Meist ist der niedrigste Isolation Level die Standardeinstellung. Sie ist für viele Anwendungen mit wenigen gleichzeitigen Benutzern ausreichend.

Mehr Aufwand bei der Isolation der Daten müssen Sie betreiben, wenn Sie ein Datenbanksystem mit vielen Tausend gleichzeitig arbeitenden Benutzern betreiben. Dann leidet die Performance der Datenbank unter einem ungeeigneten Isolation Level.

Besonderheiten bei MariaDB

Um Transaktionen verwenden zu können, müssen Sie in MariaDB den Tabellentyp `InnoDB` verwenden. Dieser ist mit Ausnahme der Systemtabellen der Standardtabellentyp.

Welche Tabellentypen von dem aktuell gestarteten MariaDB-Server unterstützt werden, erfahren Sie durch Aufruf der SQL-Anweisung `SHOW ENGINES` in der MariaDB-Konsole.

```
MariaDB [uebungen]> SHOW ENGINES;
+--------------------+---------+----------------------------------------------------------------------------------------------------------------+--------------+------+------------+
| Engine             | Support | Comment                                                                                                        | Transactions | XA   | Savepoints |
+--------------------+---------+----------------------------------------------------------------------------------------------------------------+--------------+------+------------+
| CSV                | YES     | Stores tables as CSV files                                                                                     | NO           | NO   | NO         |
| MRG_MyISAM         | YES     | Collection of identical MyISAM tables                                                                          | NO           | NO   | NO         |
| MEMORY             | YES     | Hash based, stored in memory, useful for temporary tables                                                      | NO           | NO   | NO         |
| Aria               | YES     | Crash-safe tables with MyISAM heritage. Used for internal temporary tables and privilege tables                | NO           | NO   | NO         |
| MyISAM             | YES     | Non-transactional engine with good performance and small data footprint                                        | NO           | NO   | NO         |
| SEQUENCE           | YES     | Generated tables filled with sequential values                                                                 | YES          | NO   | YES        |
| InnoDB             | DEFAULT | Supports transactions, row-level locking, foreign keys and encryption for tables                               | YES          | YES  | YES        |
| PERFORMANCE_SCHEMA | YES     | Performance Schema                                                                                             | NO           | NO   | NO         |
+--------------------+---------+----------------------------------------------------------------------------------------------------------------+--------------+------+------------+
8 rows in set (0.001 sec)
```

Der MariaDB-Tabellentyp `MyISAM` unterstützt keine Transaktionen, da dieser Tabellentyp kein Sperren auf Datenebene unterstützt.

Um Transaktionen unter MariaDB benutzen zu können, stellen Sie sicher, dass bei Installation des MariaDB-Servers die Unterstützung des `InnoDB`-Tabellentyps nicht durch den Eintrag `skip-innodb` ausgeschaltet wurde bzw. unter Linux das `configure`-Skript mit dem zusätzlichen Parameter `--with-innodb` ausgeführt wurde.

Wenn Sie eine ältere Version des Ursprungssystems von MariaDB MySQL verwenden sollten, können Sie explizit Tabellen vom Typ `InnoDB` erstellen.

- Erstellen Sie in einer Datenbank eine Tabelle vom Typ `InnoDB`.

```
CREATE TABLE tabellenname (
    feldname_1 datentyp
    ...
    feldname_n datentyp(länge),
    PRIMARY KEY (feldname)) TYPE=InnoDB;
```

oder Haben Sie nach einem System-Update noch Tabellen in Verwendung, die mit einer älteren Version erstellt wurden, wandeln Sie eine vorhandene `MyISAM`-Tabelle mit der SQL-Anweisung `ALTER TABLE` in eine Tabelle vom Typ `InnoDB` um.

```
ALTER TABLE tabellenname TYPE=INNODB;
```

Wandeln Sie keine MariaDB-Systemtabellen (`MyISAM`-Tabellentyp) in den Tabellentyp `InnoDB` um, da sonst Ihr MariaDB-System nicht mehr gestartet werden kann.

14.3 Transaktionen abschließen

Mit der Anweisung `COMMIT` schließen Sie eine Transaktion ab. Alle Änderungen an den Daten werden daraufhin endgültig durchgeführt.

- Alle Änderungen werden fest in die Datenbank geschrieben.
- Die neuen Daten werden für andere Prozesse sichtbar gemacht.
- Alle offenen Cursors werden geschlossen.

Auch wenn in der Transaktion nur `SELECT`-Abfragen verwendet wurden, sollten Sie die Transaktion mit `COMMIT` beenden, da eventuelle andere Prozesse auf den Zugriff warten.

Syntax der `COMMIT`-Anweisung

```
SET TRANSACTION ...;
...
COMMIT [TRANSACTION transaktionsname];
```

- Die Anweisung beginnt mit dem Schlüsselwort `COMMIT`.
- Einige Datenbanksysteme erwarten danach das Schlüsselwort `TRANSACTION`.
- Wurde beim Erstellen einer Transaktion ein Name angegeben, so kann gezielt diese Transaktion beendet werden. Standardmäßig wird die aktuelle Transaktion abgeschlossen.

© HERDT-Verlag

14.4 Transaktionen zurücksetzen

Transaktionen erlauben das Zurücknehmen aller Änderungen an den Daten bis zum Zustand vor der Transaktion. Dies erreichen Sie mit der ROLLBACK-Anweisung. Die Anweisung nimmt stets alle Anweisungen in ihrer Gesamtheit zurück. Es ist nicht möglich, einzelne Änderungen zu verwerfen. Wie mit der COMMIT-Anweisung wird auch mit ROLLBACK die Transaktion beendet.

Falls das Datenbanksystem aufgrund eines Fehlers abstürzt, wird beim Neustart in der Regel automatisch ein ROLLBACK ausgeführt und alle noch offenen Transaktionen werden verworfen.

Beispiel: *Rollback.sql*

Es sollen alle Artikel aus dem Datenbestand gelöscht werden, die nicht mehr am Lager sind. Dazu will ein Mitarbeiter aus der Tabelle *t_lager* alle Artikel löschen, deren Stückzahl kleiner ist als 1. Durch einen Eingabefehler löscht er jedoch alle Artikel mit einer Stückzahl größer als 1. Mit der Anweisung ROLLBACK wird die Änderung rückgängig gemacht.

```
① BEGIN;
② SELECT COUNT(id) FROM t_lager WHERE stueck < 1;
③ DELETE FROM t_lager WHERE stueck > 1;
④ ROLLBACK;
```

① Hier wird die Transaktion gestartet.

② Die Abfrage ermittelt, wie viele Artikel gelöscht werden müssen.

③ Mit der DELETE-Anweisung sollen die betreffenden Artikel gelöscht werden. Durch einen Eingabefehler wird aus der korrekten Bedingung `stueck < 1` jedoch fälschlicherweise `stueck > 1`. Dadurch werden alle Daten der gelagerten Artikel gelöscht.

④ Die Anweisung ROLLBACK macht das Löschen der Artikel wieder rückgängig, indem alle Änderungen der Anweisungen innerhalb der Transaktion zurückgesetzt werden.

In einer realen Anwendung mit grafischer Benutzeroberfläche können durch die Schaltflächen *OK* und *Abbrechen* die entsprechenden Aktionen bestätigt (*OK* – COMMIT) oder rückgängig (*Abbrechen* – ROLLBACK) gemacht werden.

Syntax der **ROLLBACK**-Anweisung

```
SET TRANSACTION ...;
...
ROLLBACK [TRANSACTION transaktionsname];
```

- Die Anweisung beginnt mit dem Schlüsselwort ROLLBACK.
- Falls beim Erstellen einer Transaktion ein Name angegeben wurde, kann mithilfe des Schlüsselworts TRANSACTION, gefolgt vom gewünschten Namen, eine Transaktion gezielt zurückgenommen werden.

© HERDT-Verlag
Schulversion

14.5 Übung

Mit Transaktionen arbeiten

Übungsdatei: -- **Ergebnisdateien: *Uebung1-Client1.sql, Uebung1-Client2.sql,Uebung2.sql***

1. Starten Sie zweimal den Datenbank-Client psql (PostgreSQL) bzw. mysql (MariaDB).

   Wechseln Sie jeweils zur Datenbank *Uebungen*.

   Starten Sie in beiden Clients eine neue Transaktion.

   Führen Sie in beiden Clients eine `SELECT`-Abfrage aus, um alle Datensätze der Tabelle *t_ lager* zu ermitteln.

   Vergleichen Sie die beiden Ergebnisse.

   Fügen Sie nun in einem Client in die Tabelle *t_lager* einen neuen Artikel mit der ID-Nummer 55, der Stückzahl 30, dem Preis 25 und der Bezeichnung 'A5 Ordner' ein.

   Führen Sie in beiden Clients wiederum die Abfrage für alle Datensätze der Tabelle aus. Was stellen Sie fest?

   Schließen Sie die erste Transaktion nach dem Einfügen der Daten ab und wiederholen Sie den vorherigen Schritt. Vergleichen Sie die Ergebnisse.

   Wird der neue Datensatz in dem anderen Client noch nicht angezeigt, führen Sie dort ebenfalls ein `COMMIT` aus und wiederholen Sie die Anzeige der Tabelle.

2. Starten Sie in der Datenbank *Uebungen* eine neue Transaktion.

   Löschen Sie alle Mitarbeiter, die aus Berlin kommen, aus der Tabelle *t_ma*.

   Führen Sie eine `SELECT`-Abfrage auf diese Tabelle aus, um zu prüfen, ob die Datensätze gelöscht wurden.

   Verwerfen Sie die Änderungen der Transaktion.

   Führen Sie die `SELECT`-Abfrage erneut aus und überprüfen Sie das Ergebnis.

Schulversion
© HERDT-Verlag